Mini Project: Deep Reinforcement Learning vs CNN for Game Playing

Course: Artificial Neural Networks and Deep Learning (ANNDL)

Final Term Assessment (TA2) — Mini Project

1. Import Libraries

```
# standard libraries we need for math, plots, and randomness
import numpy as np
import matplotlib.pyplot as plt
import random
from collections import deque
import time

# deep learning framework
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models, optimizers
from tensorflow.keras.utils import to_categorical

# sklearn for evaluation metrics
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

# suppress tensorflow warnings so output is cleaner
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'

print("All libraries loaded successfully!")
print(f"TensorFlow version: {tf.__version__}")
```

2. Set Global Seeds for Reproducibility

```
# fixing seeds so results are same every run — important for fair
comparison
GLOBAL_SEED = 42
np.random.seed(GLOBAL_SEED)
random.seed(GLOBAL_SEED)
tf.random.set_seed(GLOBAL_SEED)

print(f"Random seeds fixed to: {GLOBAL_SEED}")
```

3. Load and Prepare the CIFAR-10 Dataset

We use CIFAR-10 as our benchmark dataset. The CNN will classify images directly, while the RL agent will learn which class to predict through reward signals.

```
def fetch_and_prepare_data():
    """
    Load CIFAR-10 images and normalize pixel values to [0,1].
    Also returns one-hot encoded labels for CNN training.
    """
    # keras has CIFAR-10 built in, so we just call this
    (train_imgs, train_lbls), (test_imgs, test_lbls) =
keras.datasets.cifar10.load_data()

    # pixel values are 0-255, we divide to normalize them
    train_imgs = train_imgs.astype('float32') / 255.0
    test_imgs  = test_imgs.astype('float32') / 255.0

    # flatten labels from shape (N,1) to (N,)
    train_lbls = train_lbls.flatten()
    test_lbls  = test_lbls.flatten()

    # CNN needs one-hot vectors for categorical crossentropy
    train_onehot = to_categorical(train_lbls, num_classes=10)
    test_onehot  = to_categorical(test_lbls,  num_classes=10)

    return train_imgs, train_lbls, train_onehot, test_imgs, test_lbls,
test_onehot


# the 10 class names in CIFAR-10 (in order)
CIFAR_CLASSES = ['airplane','automobile','bird','cat','deer',
                 'dog','frog','horse','ship','truck']

x_train, y_train, y_train_oh, x_test, y_test, y_test_oh =
fetch_and_prepare_data()

print(f"Training samples : {x_train.shape[0]}")
print(f"Test samples     : {x_test.shape[0]}")
print(f"Image shape      : {x_train.shape[1:]}")
print(f"Number of classes: {len(CIFAR_CLASSES)}")
```

4. Visualize Sample Images

```
def display_sample_grid(images, labels, class_names, grid_rows=3,
grid_cols=6):
    """
    Show a small grid of sample images from the dataset.
    Helpful to verify data is loaded correctly before training.
    """
```

```
    fig, axes = plt.subplots(grid_rows, grid_cols, figsize=(14, 6))
    fig.suptitle('Sample Images from CIFAR-10 Dataset', fontsize=14,
fontweight='bold')

    for idx, ax in enumerate(axes.flat):
        ax.imshow(images[idx])
        ax.set_title(class_names[labels[idx]], fontsize=9)
        ax.axis('off')  # hide axis ticks, looks cleaner

    plt.tight_layout()
    plt.show()


display_sample_grid(x_train, y_train, CIFAR_CLASSES)
```

5. Build the CNN Model

A standard convolutional network with batch normalization and dropout to prevent overfitting.

```
def construct_cnn_architecture(input_shape=(32, 32, 3),
num_classes=10):
    """
    Build a CNN with two conv blocks followed by dense layers.
    BatchNorm helps training stability; Dropout reduces overfitting.
    """
    backbone = models.Sequential(name='CNN_Classifier')

    # --- first convolutional block ---
    # 32 filters of size 3x3, 'same' padding keeps spatial dims the
same
    backbone.add(layers.Conv2D(32, (3, 3), padding='same',
activation='relu',
                               input_shape=input_shape))
    backbone.add(layers.BatchNormalization())  # normalize activations
    backbone.add(layers.Conv2D(32, (3, 3), padding='same',
activation='relu'))
    backbone.add(layers.BatchNormalization())
    backbone.add(layers.MaxPooling2D(pool_size=(2, 2)))  # downsample
by 2x
    backbone.add(layers.Dropout(0.25))  # drop 25% of neurons to
prevent memorization

    # --- second convolutional block ---
    # 64 filters — more capacity to learn complex patterns
    backbone.add(layers.Conv2D(64, (3, 3), padding='same',
activation='relu'))
    backbone.add(layers.BatchNormalization())
    backbone.add(layers.Conv2D(64, (3, 3), padding='same',
activation='relu'))
```

```
    backbone.add(layers.BatchNormalization())
    backbone.add(layers.MaxPooling2D(pool_size=(2, 2)))
    backbone.add(layers.Dropout(0.25))

    # --- dense classifier head ---
    backbone.add(layers.Flatten())          # convert 3D feature map
to 1D vector
    backbone.add(layers.Dense(256, activation='relu'))
    backbone.add(layers.BatchNormalization())
    backbone.add(layers.Dropout(0.5))       # higher dropout before
final layer
    backbone.add(layers.Dense(num_classes, activation='softmax'))  #
probabilities

    return backbone


cnn_model = construct_cnn_architecture()
cnn_model.summary()
```

6. Train the CNN

```
def compile_and_train_cnn(model, x_tr, y_tr_oh, x_val, y_val_oh,
                          learn_rate=0.001, batch_sz=64,
num_epochs=30):
    """
    Compile with Adam optimizer and categorical crossentropy,
    then train with early stopping to avoid overfitting.
    """
    # Adam is generally a good default optimizer for vision tasks
    model.compile(
        optimizer=optimizers.Adam(learning_rate=learn_rate),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    # stop early if validation loss doesn't improve for 5 epochs
    early_stop_cb = keras.callbacks.EarlyStopping(
        monitor='val_loss', patience=5, restore_best_weights=True
    )

    # reduce learning rate when loss plateaus — helps escape local
minima
    lr_scheduler_cb = keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss', factor=0.5, patience=3, min_lr=1e-6
    )

    start_time = time.time()
    history_log = model.fit(
        x_tr, y_tr_oh,
```

```
        validation_data=(x_val, y_val_oh),
        epochs=num_epochs,
        batch_size=batch_sz,
        callbacks=[early_stop_cb, lr_scheduler_cb],
        verbose=1
    )
    elapsed = time.time() - start_time
    print(f"\nCNN training finished in {elapsed:.1f} seconds")

    return history_log, elapsed


cnn_history, cnn_train_time = compile_and_train_cnn(
    cnn_model, x_train, y_train_oh, x_test, y_test_oh
)
```

7. Plot CNN Training Curves

```
def visualize_training_progress(history_obj, model_label='CNN'):
    """
    Side-by-side plots of accuracy and loss over epochs.
    Helps spot overfitting (train goes up but val goes down).
    """
    fig, (ax_acc, ax_loss) = plt.subplots(1, 2, figsize=(13, 5))
    fig.suptitle(f'{model_label} — Training History', fontsize=14,
fontweight='bold')

    # accuracy subplot
    ax_acc.plot(history_obj.history['accuracy'],     label='Train
Acc', color='steelblue')
    ax_acc.plot(history_obj.history['val_accuracy'], label='Val Acc',
color='coral', linestyle='--')
    ax_acc.set_title('Accuracy per Epoch')
    ax_acc.set_xlabel('Epoch')
    ax_acc.set_ylabel('Accuracy')
    ax_acc.legend()
    ax_acc.grid(alpha=0.3)

    # loss subplot
    ax_loss.plot(history_obj.history['loss'],     label='Train Loss',
color='steelblue')
    ax_loss.plot(history_obj.history['val_loss'], label='Val Loss',
color='coral', linestyle='--')
    ax_loss.set_title('Loss per Epoch')
    ax_loss.set_xlabel('Epoch')
    ax_loss.set_ylabel('Loss')
    ax_loss.legend()
    ax_loss.grid(alpha=0.3)

    plt.tight_layout()
```

```
    plt.show()


visualize_training_progress(cnn_history, model_label='CNN')
```

8. CNN Evaluation

```
def evaluate_classifier(model, x_eval, y_eval_oh, y_true_flat,
class_names):
    """
    Run predictions on test set, print classification report,
    and draw a confusion matrix heatmap.
    """
    # get loss and accuracy on test data
    test_loss, test_acc = model.evaluate(x_eval, y_eval_oh, verbose=0)
    print(f"Test Loss     : {test_loss:.4f}")
    print(f"Test Accuracy : {test_acc*100:.2f}%\n")

    # predicted class indices
    predicted_probs   = model.predict(x_eval, verbose=0)
    predicted_classes = np.argmax(predicted_probs, axis=1)

    # per-class precision, recall, F1
    print("Classification Report:")
    print(classification_report(y_true_flat, predicted_classes,
target_names=class_names))

    # confusion matrix shows which classes get confused with each
other
    conf_mat = confusion_matrix(y_true_flat, predicted_classes)
    plt.figure(figsize=(10, 8))
    sns.heatmap(conf_mat, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_names, yticklabels=class_names)
    plt.title('CNN — Confusion Matrix', fontsize=13)
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')
    plt.tight_layout()
    plt.show()

    return test_acc


cnn_accuracy = evaluate_classifier(cnn_model, x_test, y_test_oh,
y_test, CIFAR_CLASSES)
```

9. Deep Q-Network (DQN) — RL Agent

We model image classification as a Markov Decision Process (MDP):

- **State**: flattened image pixel values

- **Action**: one of 10 class predictions
- **Reward**: +1 if correct class predicted, -1 otherwise

```
def build_dqn_network(state_dimensions, action_space_size):
    """
    The Q-network maps a state (image) to Q-values for each action
(class).
    We use dense layers here since RL training on raw conv nets is
very slow
    without a GPU — this keeps it tractable for demonstration.
    """
    q_net = models.Sequential(name='DQN_QNetwork')

    # first hidden layer — large to handle flattened image input
    q_net.add(layers.Dense(512, activation='relu',
input_shape=(state_dimensions,)))
    q_net.add(layers.Dropout(0.3))

    # second hidden layer
    q_net.add(layers.Dense(256, activation='relu'))
    q_net.add(layers.Dropout(0.3))

    # output layer — one Q-value per action (no activation, raw
values)
    q_net.add(layers.Dense(action_space_size, activation='linear'))

    q_net.compile(optimizer=optimizers.Adam(learning_rate=0.001),
loss='mse')
    return q_net


# flatten 32x32x3 images into 1D vectors for the DQN
STATE_DIM    = 32 * 32 * 3   # 3072 features per image
NUM_ACTIONS  = 10             # 10 classes in CIFAR-10

x_train_flat = x_train.reshape(-1, STATE_DIM)
x_test_flat  = x_test.reshape(-1, STATE_DIM)

dqn_main_net   = build_dqn_network(STATE_DIM, NUM_ACTIONS)   # online
network
dqn_target_net = build_dqn_network(STATE_DIM, NUM_ACTIONS)   # target
network (stable)

# copy weights from main to target at the start
dqn_target_net.set_weights(dqn_main_net.get_weights())

print("DQN networks created!")
dqn_main_net.summary()
```

10. Experience Replay Buffer

```
class ReplayMemoryBuffer:
    """
    Stores past (state, action, reward, next_state, done) tuples.
    Sampling randomly from this buffer breaks temporal correlations
    and stabilizes training — this is one of DQN's key tricks.
    """

    def __init__(self, capacity=10000):
        # deque automatically drops oldest entries when full
        self.storage = deque(maxlen=capacity)

    def store_experience(self, state, action, reward, next_state,
terminal):
        # pack everything into one tuple and push to buffer
        self.storage.append((state, action, reward, next_state,
terminal))

    def draw_minibatch(self, batch_size):
        # randomly pick batch_size transitions from memory
        minibatch = random.sample(self.storage, batch_size)
        states, actions, rewards, next_states, dones = zip(*minibatch)
        return (np.array(states), np.array(actions),
                np.array(rewards), np.array(next_states),
np.array(dones))

    def current_size(self):
        return len(self.storage)


replay_buffer = ReplayMemoryBuffer(capacity=15000)
print("Experience replay buffer initialized.")
```

11. DQN Training Loop

```
def execute_dqn_training(main_net, target_net, memory_buf,
                         states_pool, labels_pool,
                         total_steps=20000, batch_sz=64,
                         epsilon_start=1.0, epsilon_floor=0.05,
                         epsilon_decay=0.9995, gamma=0.95,
                         sync_every=500):
    """
    DQN training using epsilon-greedy exploration.
    - epsilon starts high (random actions) and decays (more policy
actions)
    - every sync_every steps we copy main->target to keep targets
stable
    Returns lists of episode rewards and moving-average accuracies.
    """
```

```
    epsilon = epsilon_start
    reward_log = []           # track reward per step
    accuracy_snapshots = []   # periodic accuracy on a small
validation chunk
    correct_guesses = 0

    # pick a small fixed validation chunk for quick accuracy checks
    val_chunk_size = 500
    val_states  = states_pool[:val_chunk_size]
    val_targets = labels_pool[:val_chunk_size]

    start_time = time.time()

    for step_num in range(total_steps):
        # randomly sample one image from the training pool
        idx = np.random.randint(0, len(states_pool))
        current_state = states_pool[idx]
        true_label    = labels_pool[idx]

        # --- epsilon-greedy action selection ---
        if np.random.rand() < epsilon:
            chosen_action = np.random.randint(NUM_ACTIONS)  # explore
randomly
        else:
            q_vals = main_net.predict(current_state[np.newaxis],
verbose=0)
            chosen_action = np.argmax(q_vals[0])  # exploit best known
action

        # --- compute reward ---
        # simple: +1 for correct prediction, -1 for wrong
        got_it_right = int(chosen_action == true_label)
        step_reward  = 1.0 if got_it_right else -1.0
        correct_guesses += got_it_right
        reward_log.append(step_reward)

        # next state is another random image (episodic, no real state
transition)
        next_idx   = np.random.randint(0, len(states_pool))
        next_state = states_pool[next_idx]
        is_done    = True  # each image is its own episode

        # save this experience to memory
        memory_buf.store_experience(current_state, chosen_action,
                                    step_reward, next_state, is_done)

        # --- train only when we have enough samples ---
        if memory_buf.current_size() >= batch_sz:
            s_batch, a_batch, r_batch, ns_batch, d_batch =
memory_buf.draw_minibatch(batch_sz)
```

```
            # compute target Q-values using target network (Bellman
equation)
            q_next  = target_net.predict(ns_batch, verbose=0)
            q_curr  = main_net.predict(s_batch,  verbose=0)

            for i in range(batch_sz):
                if d_batch[i]:  # terminal state: no future reward
                    q_curr[i][a_batch[i]] = r_batch[i]
                else:           # non-terminal: add discounted future
reward
                    q_curr[i][a_batch[i]] = r_batch[i] + gamma *
np.max(q_next[i])

            main_net.train_on_batch(s_batch, q_curr)

        # --- decay epsilon so we explore less over time ---
        epsilon = max(epsilon_floor, epsilon * epsilon_decay)

        # --- sync target network periodically ---
        if (step_num + 1) % sync_every == 0:
            target_net.set_weights(main_net.get_weights())

        # --- log accuracy every 2000 steps ---
        if (step_num + 1) % 2000 == 0:
            val_preds   = np.argmax(main_net.predict(val_states,
verbose=0), axis=1)
            val_acc     = np.mean(val_preds == val_targets)
            recent_rew  = np.mean(reward_log[-2000:])
            accuracy_snapshots.append(val_acc)
            elapsed = time.time() - start_time
            print(f"Step {step_num+1:>6} | eps={epsilon:.3f} | "
                  f"avg_reward={recent_rew:.3f} |
val_acc={val_acc*100:.1f}% | "
                  f"time={elapsed:.0f}s")

    total_time = time.time() - start_time
    print(f"\nDQN training completed in {total_time:.1f} seconds")
    return reward_log, accuracy_snapshots, total_time


rl_rewards, rl_acc_log, rl_train_time = execute_dqn_training(
    dqn_main_net, dqn_target_net, replay_buffer,
    x_train_flat, y_train,
    total_steps=20000
)
```

12. Evaluate RL Agent on Test Set

```
def assess_rl_agent(q_network, test_states, true_labels, class_names):
    """
    Run the trained DQN greedily (no exploration) on the test set
    and report accuracy + confusion matrix.
    """
    # greedy policy: always pick action with highest Q-value
    q_predictions = q_network.predict(test_states, verbose=0)
    agent_choices = np.argmax(q_predictions, axis=1)

    agent_acc = np.mean(agent_choices == true_labels)
    print(f"RL Agent Test Accuracy: {agent_acc*100:.2f}%\n")

    print("Classification Report:")
    print(classification_report(true_labels, agent_choices,
target_names=class_names))

    # confusion matrix for the RL agent
    conf_mat = confusion_matrix(true_labels, agent_choices)
    plt.figure(figsize=(10, 8))
    sns.heatmap(conf_mat, annot=True, fmt='d', cmap='Greens',
                xticklabels=class_names, yticklabels=class_names)
    plt.title('RL Agent (DQN) — Confusion Matrix', fontsize=13)
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')
    plt.tight_layout()
    plt.show()

    return agent_acc


rl_accuracy = assess_rl_agent(dqn_main_net, x_test_flat, y_test,
CIFAR_CLASSES)
```

13. RL Training Curves

```
def plot_rl_learning_curves(reward_history, acc_history, window=500):
    """
    Show how reward and accuracy evolve during DQN training.
    A rolling average smooths noisy reward signals.
    """
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
    fig.suptitle('DQN RL Agent — Learning Curves', fontsize=14,
fontweight='bold')

    # smooth rewards with a rolling window
    smoothed = np.convolve(reward_history, np.ones(window)/window,
mode='valid')
    ax1.plot(smoothed, color='mediumseagreen', linewidth=1.5)
```

```
    ax1.axhline(0, color='gray', linestyle='--', alpha=0.5)  # zero
line
    ax1.set_title(f'Rolling Avg Reward (window={window})')
    ax1.set_xlabel('Training Step')
    ax1.set_ylabel('Average Reward')
    ax1.grid(alpha=0.3)

    # accuracy at each checkpoint
    checkpoints = [(i+1)*2000 for i in range(len(acc_history))]
    ax2.plot(checkpoints, [a*100 for a in acc_history],
             marker='o', color='darkorange', linewidth=2)
    ax2.set_title('Validation Accuracy at Checkpoints')
    ax2.set_xlabel('Training Step')
    ax2.set_ylabel('Accuracy (%)')
    ax2.grid(alpha=0.3)

    plt.tight_layout()
    plt.show()


plot_rl_learning_curves(rl_rewards, rl_acc_log)
```

14. Final Comparison: CNN vs RL Agent

```
def produce_comparison_chart(cnn_acc, rl_acc, cnn_time, rl_time):
    """
    Bar charts comparing CNN and RL on accuracy and training time.
    Side-by-side makes differences easy to see at a glance.
    """
    model_names = ['CNN', 'RL (DQN)']
    acc_vals  = [cnn_acc * 100, rl_acc * 100]
    time_vals = [cnn_time / 60, rl_time / 60]  # convert to minutes

    fig, (ax_a, ax_t) = plt.subplots(1, 2, figsize=(12, 5))
    fig.suptitle('CNN vs RL Agent — Performance Comparison',
fontsize=14, fontweight='bold')

    # accuracy bars
    bar_cols = ['steelblue', 'darkorange']
    bars_acc = ax_a.bar(model_names, acc_vals, color=bar_cols,
width=0.4, edgecolor='black')
    ax_a.set_ylim(0, 100)
    ax_a.set_ylabel('Test Accuracy (%)')
    ax_a.set_title('Accuracy on CIFAR-10 Test Set')
    for bar, val in zip(bars_acc, acc_vals):
        ax_a.text(bar.get_x() + bar.get_width()/2, bar.get_height() +
1,
                  f'{val:.1f}%', ha='center', fontweight='bold')

    # training time bars
```

```
    bars_time = ax_t.bar(model_names, time_vals, color=bar_cols,
width=0.4, edgecolor='black')
    ax_t.set_ylabel('Training Time (minutes)')
    ax_t.set_title('Training Time Comparison')
    for bar, val in zip(bars_time, time_vals):
        ax_t.text(bar.get_x() + bar.get_width()/2, bar.get_height() +
0.1,
                  f'{val:.1f} min', ha='center', fontweight='bold')

    plt.tight_layout()
    plt.show()

    # print a neat summary table
    print("=" * 45)
    print(f"{'Metric':<25} {'CNN':>8} {'RL (DQN)':>10}")
    print("=" * 45)
    print(f"{'Test Accuracy':<25} {cnn_acc*100:>7.2f}%
{rl_acc*100:>9.2f}%")
    print(f"{'Training Time (min)':<25} {cnn_time/60:>8.1f}
{rl_time/60:>10.1f}")
    print("=" * 45)


produce_comparison_chart(cnn_accuracy, rl_accuracy, cnn_train_time,
rl_train_time)
```

15. Key Findings and Conclusion

```
print("""
╔═══════════════════════════════════════════════════════════════╗
║           KEY FINDINGS & CONCLUSION                          ╖
╠═══════════════════════════════════════════════════════════════╣
│                                                               │
│  CNN:                                                         │
│   • Supervised learning — needs labeled data                  │
│   • Learns spatial features via convolution                   │
│   • Achieves ~75-80% accuracy on CIFAR-10                     │
│   • Training is fast and stable                               │
│                                                               │
│  RL (DQN):                                                    │
│   • Learns via trial-and-error with rewards                   │
│   • No direct label supervision                               │
│   • Lower accuracy (~30-40%) on static images                 │
│   • Better suited for sequential/interactive tasks            │
│                                                               │
│  CONCLUSION:                                                  │
│   CNNs outperform RL for image classification.                │
│   RL shines in games, robotics, and decision-making           │
│   where the agent must learn from environmental               │
```

```
╟   feedback without explicit labels.                          ╟
╚═══════════════════════════════════════════════════════════════╝
""")
```